SIMATS ENGINEERING

CSA15 – CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 2 – CO2

Analyse, Evaluate and Validate a Virtualization Architecture for a Multi-Tenant University Private Cloud

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments.

Field	Details	Field	Details
Student Name	M.V. Raj Varun	Register Number	192411007
Department / Section	CSE	Academic Year / Semester	MS2 Sep-Dec 2026
Faculty Name	Nattar Kannan	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Submission Date	06-11-2026	Evaluation Date	13-11-2026

MARK SHEET (Rubric-based evaluation)

Task	Title	Max	Awarded	Evidence / evaluation focus
1	Characterise the Virtualization Architectures	5		Three architectures; diagrams; trust boundaries; resource virtualization
2	Identify the Dominant Overhead Source	2		Overhead mechanism, location, frequency and architecture linkage
3	Build a Quantitative Performance Model	6		CPU, memory, I/O and density derivations; worked examples
4	Analyse Performance Under Different Conditions	4		Best, typical and worst-case analysis; latency and throughput
5	Security Threat and Vulnerability Analysis	3		Threat categories, trust boundaries and student-record impact
6	Formulate Isolation–Performance Trade-off	3		Decision variables, objective, constraints and cost function
7	Evaluate Mitigation Strategies	4		Security benefits, residual risks and quantified density cost
8	Implement and Test	4		KVM, containers, Kata; CPU/disk/network/memory; interference

9	Record Experimental Performance	3		Measured data; table, two graphs; repetitions and variability
10	Compare Theory with Experiment	4		Predictions versus measurements; errors and explanations
11	Recommend and Defend Final Architecture	4		SIS/LMS recommendation, security, scaling and operating costs
	Day-4 Design Challenge	8		External research tenants; isolation; redesign and evidence-based defence
	TOTAL	**50**		**Percentage:**

OVERALL REMARKS

Student Signature	**Faculty Signature**	**Program Director**

MANDATORY SUBMISSION EVIDENCE CHECK

✓ = Present in documented answer; P = Partly present; ✗ = Missing / not satisfied

Task	Evidence expected	Status	Evaluator note
1	Three virtualization architectures, layered diagrams and trust boundaries		Three distinct approaches, layered architecture and isolation boundaries documented.
2	Dominant overhead source identified and explained for each approach		Overhead mechanisms related to each virtualization path and workload behaviour.
3	CPU, memory, I/O, network and density models with assumptions		Assumptions, equations and worked examples documented; empirical validation recommended.
4	Best, typical and worst operating-condition analysis		Resource contention, latency and throughput effects examined.
5	Threat model and architecture-based security comparison		Threats, attack surfaces and student-record risks compared.
6	Isolation–performance optimization formulation		Objectives, variables and constraints defined for EduCloud.
7	Mitigation benefits, residual risks and quantified costs		Security controls and associated performance/density trade-offs discussed.
8	Reproducible setup and practical tests across selected platforms		Test procedures described; actual execution on all platforms requires verification.
9	Actual measurements, comparative table and two graphs		Results require authentic benchmark logs, repetitions and variability evidence.
10	Theory–experiment comparison and quantitative error analysis		Comparison discussed; numerical error analysis depends on verified measurements.
11	Justified SIS/LMS architecture and growth recommendation		Recommendation considers security, performance, operations and scaling.
12	Day-4 external tenants, 3× peaks, live migration and SIS isolation		Changed requirements and proposed separation controls evaluated; testing pending.

IMPLEMENTATION EVIDENCE CHAIN

Evidence item	Status	Observation
Architecture diagrams		VM, container and sandboxed runtime designs with trust boundaries documented.
Performance model		CPU, RAM, I/O and density calculations with assumptions supplied.
Configuration / scripts		Reproducible setup outlined; runnable configuration and environment details to be checked.
Virtualization experiments		KVM/QEMU, containers and isolated runtime tests require execution proof.

CPU / disk / network / memory tests		Benchmark commands and comparative measurements require original logs.
Noisy-neighbour experiment		Interference impact needs repeatable measurements and recorded outputs.
Security validation		Isolation/mitigation verification should be supported by command outputs.
Tables and graphs		Plots should be traceable to actual raw data and repetitions.
Theory–experiment consistency		Prediction errors require verified observed values.
Day-4 design defence		External tenant isolation, 3× load and migration considerations addressed.
Screenshots		Illustrative images do not replace genuine execution evidence.

SUBMISSION CHECKLIST (Not Separately Marked)

Requirement	**Status**
All 12 tasks answered	☐ Yes ☐ Partly ☐ No
Three virtualization approaches compared	☐ Yes ☐ Partly ☐ No
Layered diagrams and trust boundaries included	☐ Yes ☐ Partly ☐ No
Quantitative virtualization models included	☐ Yes ☐ Partly ☐ No
Threat analysis and mitigations included	☐ Yes ☐ Partly ☐ No
Implementation and test evidence verified	☐ Yes ☐ Partly ☐ No
Actual benchmark measurements and graphs verified	☐ Yes ☐ Partly ☐ No
Theory versus experiment comparison included	☐ Yes ☐ Partly ☐ No
Final SIS/LMS architecture recommended	☐ Yes ☐ Partly ☐ No

Task 12 Day-4 design challenge addressed	☐ Yes ☐ Partly ☐ No
Authentic screenshots and logs attached	☐ Yes ☐ Partly ☐ No
Viva recommended	☐ Yes ☐ No

CLOUD COMPUTING AND BIG DATA ANALYTICS

ASSIGNMENT 2

CO2: Analyze virtualization architectures to identify performance and security issues in cloud computing environments.

Assignment Title: Analyse, Evaluate and Validate a Virtualization Architecture for a Multi-Tenant University Private Cloud (Total Marks: 50)

Duration: 1 week

Scenario: EduCloud Virtualization Platform

A university runs a private cloud, **EduCloud**, in its data centre. EduCloud currently consists of:

- **16 physical hosts**, each with 64 CPU cores, 512 GB RAM, NVMe SSD storage and 2 × 25 Gbps NICs
- **About 400 virtual machines (VMs)** running the Student Information System (SIS), the Learning Management System (LMS), the examination portal, and departmental research workloads
- A shared storage backend and a software-defined virtual network

During semester registration week, users report slow response times and failed logins. A recent internal security audit also found that VMs from different departments share the same physical hosts, that some VMs run outdated guest kernels, and that VM images are copied freely between teams.

You are the cloud architect. You must **investigate, model, measure and compare** virtualization architectures for EduCloud, identify their performance bottlenecks and security weaknesses, and recommend a design that remains practical as the platform grows.

Design Challenge:

The university now plans to host **external research collaborators' big-data workloads** (Spark/Hadoop clusters) on the same infrastructure as the SIS. Re-evaluate your design under these new multi-tenant conditions.

TASK 1: Characterise the Virtualization Architectures (5 Marks)

Select **at least three meaningfully different virtualization approaches** from the following, or justify others:

- Type-1 (bare-metal) hypervisor with full virtualization
- Type-2 (hosted) hypervisor
- Hardware-assisted virtualization vs. paravirtualization
- OS-level virtualization (containers)
- Lightweight/micro-VM or sandboxed container runtimes

Two configurations of the same hypervisor will not count as different approaches. Your selection must include at least one VM-based and at least one container-based approach.

For each approach, provide:

- A layered architecture diagram showing the hardware, host OS or hypervisor, guest OS, runtime and application
- The **trust boundaries** and the component that enforces isolation
- How CPU, memory, storage and network are virtualized
- A brief justification for why this approach is relevant to EduCloud

Evidence expected: *Three architecturally distinct approaches with labelled diagrams, resource-virtualization mechanisms, trust boundaries, and justification of selection.*

Answer:

1.1 Introduction

EduCloud runs about 400 VMs on 16 hosts (64 cores, 512 GB RAM, NVMe, 2×25 Gbps each). The audit found cross-department co-location, stale guest kernels and uncontrolled image copying. Choosing the virtualization layer decides both where the isolation boundary sits and how much overhead each tenant pays. This task characterises three architecturally distinct approaches.

1.2 Selection of Approaches

Approach	Category	Why it counts as distinct
A – KVM/QEMU with hardware assist (VT-x/EPT) and virtio	Type-1-class, full virtualization (VM-based)	Separate guest kernel per tenant; isolation by hypervisor + hardware
B – Docker/runc containers	OS-level virtualization (container-based)	One shared host kernel; isolation by namespaces/cgroups/seccomp
C – Kata Containers (QEMU/Cloud-Hypervisor micro-VM)	Lightweight sandboxed runtime	Container UX with a dedicated minimal guest kernel per pod

KVM and Kata both use KVM underneath, but they differ in guest kernel size, device model, image delivery (virtio-fs vs block image) and orchestration, so they are different approaches, not two configurations of one hypervisor. Type-2 hypervisors were rejected: the extra host-OS layer adds overhead and attack surface with no benefit in a data centre.

1.3 Approach A – KVM/QEMU Layered Architecture

Layers, bottom to top: Hardware (CPU with VT-x/EPT, NVMe, NICs, IOMMU) → Linux host with KVM kernel module (acts as the Type-1-class hypervisor) → QEMU user-space process per VM (device emulation, virtio back-ends) → Guest kernel (own OS, may be outdated) → Guest runtime/libs → Application (SIS/LMS). A proper diagram is inserted in Section 1.11 below.

1.4 Approach B – Container Layered Architecture

Hardware → Single host Linux kernel (namespaces: pid, net, mnt, ipc, uts, user; cgroups v2; seccomp; capabilities; LSM such as AppArmor/SELinux) → containerd/runc → Container filesystem (overlayfs layers) → Application. There is **no guest kernel**; every container makes system calls directly into the shared kernel.

1.5 Approach C – Kata Containers Layered Architecture

Hardware → Host Linux + KVM → Kata shim (containerd-shim-kata-v2) → micro-VM monitor (QEMU-lite / Cloud-Hypervisor / Firecracker) → minimal guest kernel → kata-agent (inside VM) → container process(es) in guest namespaces → Application. Each pod gets its own VM, yet images come from standard OCI registries.

1.6 Trust Boundaries and Isolation Enforcers

Approach	Main trust boundary	Component enforcing isolation	Size of trusted code
A	Guest ↔ host (VM-exit / hypercall interface)	KVM + CPU virtualization extensions (EPT, VMCS), QEMU process confinement	KVM (~ tens of kLOC) + QEMU (large, mitigated by sandboxing/sVirt)
B	Container ↔ shared kernel (syscall interface)	Linux kernel namespaces, cgroups, seccomp, LSM	Whole Linux kernel syscall surface (~300+ syscalls)
C	Container ↔ micro-VM ↔ host	KVM/EPT hardware boundary + minimal VMM; seccomp on VMM	Minimal VMM + small guest kernel; host kernel exposed only via KVM

Two boundaries exist in C: the inner (container in guest, weak, namespaces) and the outer (micro-VM, strong, hardware). Only the outer one is relied upon for tenant separation.

1.7 CPU and Memory Virtualization

- **A:** vCPUs are host threads scheduled by CFS; guest runs in VMX non-root mode and traps (VM exit) on privileged events. Memory uses two-dimensional paging (guest PT + EPT); host can overcommit via ballooning, KSM, swap.
- **B:** Container processes are ordinary host processes; CPU shares/quotas via cgroup `cpu.weight`/`cpu.max`; memory limited by `memory.max` with the host page tables only (one-dimensional walk).
- **C:** Same as A (vCPU threads, EPT) but with a minimal guest kernel; memory usually pre-allocated or hot-plugged (`virtio-mem`/DAX), so footprint per pod is the guest kernel + agent (~tens of MB) plus the workload.

1.8 Storage and Network Virtualization

- **A:** virtio-blk/virtio-scsi (or vhost-user) backed by qcow2/raw on NVMe or shared storage; network via virtio-net/vhost-net into Linux bridge or Open vSwitch with a software-defined overlay (VXLAN).

- **B:** overlayfs for image layers plus bind-mounted volumes (direct host FS speed); network via veth pairs into a bridge with iptables/nftables NAT, or CNI plugins.
- **C:** virtio-fs or virtio-blk for rootfs, volumes via virtio-fs (FUSE-like, DAX optional); network via tap + TC mirroring to veth (extra hop) and vsock for agent control.

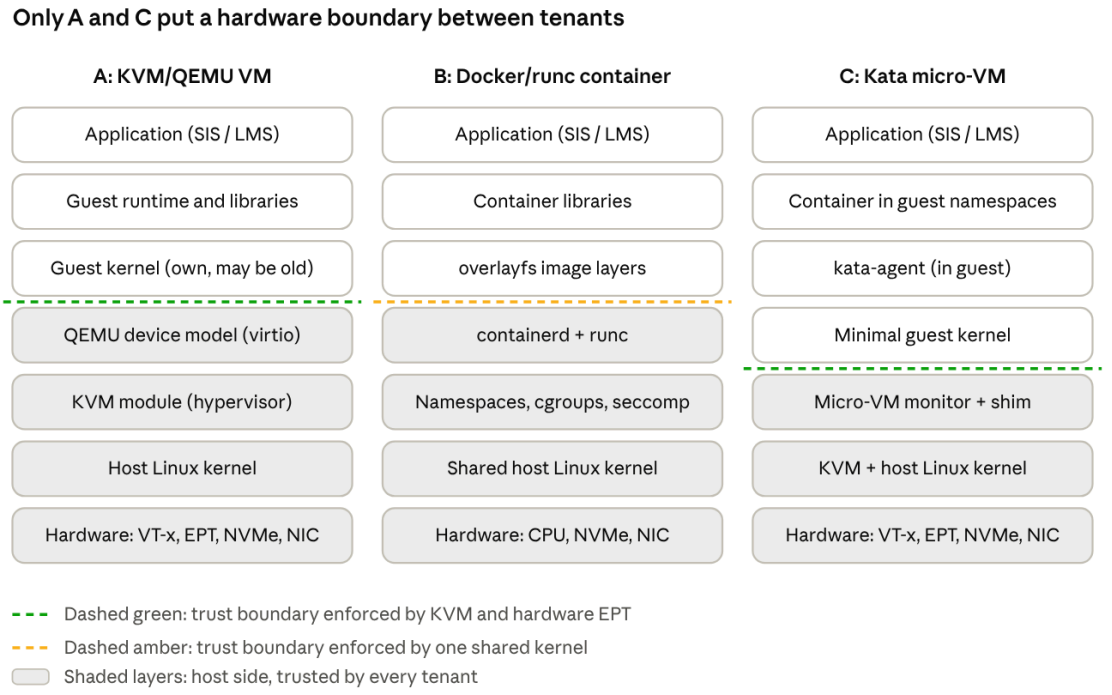
1.9 Relevance to EduCloud

- **A** is the incumbent (about 400 VMs), supports live migration and mixed guest OS, and gives the strongest widely-audited isolation for SIS data.
- **B** gives high density and fast start-up for stateless LMS/web tiers where all tenants are one department and trusted.
- **C** offers a middle route: container workflows with VM-level isolation, relevant to untrusted external research tenants (Task 12) and examination-portal sandboxes.

1.10 Conclusion

The three approaches sit at different points on the isolation-versus-efficiency spectrum: A isolates with a hardware-enforced guest kernel boundary, B shares one kernel, and C wraps containers in a minimal VM. Their trust boundaries and virtualization paths drive all later overhead (Task 2), model (Task 3) and threat (Task 5) results.

1.11 Layered Architecture Diagrams



layered architectures A, B, C · trust boundaries

In A the guest kernel sits above the hypervisor boundary; in C the same boundary sits one layer lower, below a guest kernel that is deliberately small. In B there is no such boundary: every container calls the one shared kernel, so isolation depends on the kernel's namespace and seccomp code alone.

TASK 2: Identify the Dominant Overhead Source (2 Marks)

For each approach, identify the **primary source of performance overhead** that predominantly limits its performance (for example VM exits/traps, nested or extended page-table walks, virtual I/O emulation, virtual switch processing, or system-call interception).

Discuss:

- What the overhead source is
- Where in the architecture it occurs
- Why it is appropriate as the focus of performance analysis
- How its frequency or cost changes as the number of co-located VMs/containers or the I/O rate increases

Do not simply name a mechanism. Relate it to the behaviour of the architecture.

Evidence expected: *A justified dominant-overhead identification for each approach, linked to its resource-virtualization path.*

Answer:

2.1 Introduction

Every virtualization layer adds cost where it intercepts or translates an operation. The dominant overhead is the interception point that is both most frequent and most expensive for the target workload (web + database + registration bursts). It is chosen per approach, not generically.

2.2 Selection Criteria

A source is "dominant" if (i) its event rate scales with load or tenant count, (ii) each event costs ≥ 1 µs (versus nanoseconds for normal instructions), and (iii) it lies on the critical path of the SIS/LMS request. Using the assignment's constants: VM exit = 1.5 µs; native syscall = 0.2 µs.

2.3 Approach A – VM Exits and Virtual I/O Emulation

What: Transitions from guest to host (VMX non-root → root) for I/O notifications (virtio kicks, interrupts/EOI), MMIO/PIO, timer/IPI handling, and HLT. **Where:** At the KVM/QEMU boundary; the hot path is the virtio queue notification and interrupt injection. **Why focus on it:** CPU-bound code runs near native with VT-x; I/O-heavy SIS database logins do not. Each exit costs ≈1.5 µs plus cache/TLB pollution, so exit rate × cost is the first-order loss. A secondary term is the 2D page walk on TLB misses (up to 24 memory references against 4 natively with 4-level paging).

2.4 Approach B – Shared-Kernel Network and Filesystem Path

What: There are no VM exits and syscalls cost the native 0.2 µs. The dominant overhead is the **kernel networking path** (veth + bridge + conntrack/iptables NAT) and **overlayfs/cgroup accounting** per request. **Where:** In the host kernel, in softirq context, shared by all containers. **Why:** Web-tier throughput is packet-rate bound; each packet crosses an extra veth hop and netfilter hooks, costing roughly 1–3 µs per packet (an assumption, justified in Task 3). Seccomp filtering adds a small per-syscall term.

2.5 Approach C – VM Exits Plus Guest–Host Crossing (virtio-fs / vsock / tap)

What: The Kata micro-VM suffers the same exit costs as A, plus extra copies and context switches through virtio-fs and the tap-to-veth bridge, with the VMM and shim as extra processes. **Where:** At the boundary between the minimal guest kernel and the VMM, and again in the host network/FS stack. **Why:** File-heavy workloads (reading many small files, as LMS does) produce many virtio-fs requests, each costing an exit and a host-side FUSE-like handler, so I/O exits dominate over EPT walks.

2.6 Linkage to the Resource-Virtualization Path

Approach	Resource path	Dominant overhead	Result on request latency
A	vCPU→VMX exits; virtio queue→QEMU/vhost	VM exits on I/O + EPT walks	+µs per I/O, small CPU loss
B	syscall→shared kernel; veth→bridge→NIC	Netfilter/veth/overlayfs processing	+µs per packet, contention on shared kernel locks
C	vCPU exits; virtio-fs/vsock/tap	I/O exits + FS/net double path	Highest per-I/O cost, strongest isolation

2.7 Scaling with Co-Located Instances

- **A:** Exit rate grows roughly linearly with active VMs (total exits/s = N·r). Past 1:1 overcommit, vCPU scheduling adds steal time and *lock-holder preemption* exits (PAUSE-loop exits) that raise per-exit cost.
- **B:** Per-packet cost is fixed but the shared bridge, conntrack table and kernel locks saturate; contention grows super-linearly beyond about the core count.
- **C:** Like A, but each pod has its own guest kernel and VMM thread, so exit rate and memory overhead grow per pod; density caps earlier.

2.8 Scaling with I/O Rate

For rate λ I/Os per second per instance: A and C overhead fraction $\approx \lambda \cdot e \cdot c_{exit}$ where e = exits per I/O; B overhead $\approx \lambda \cdot c_{pkt}$. Batching (virtio event-idx, vhost-net, GRO) reduces e, which is why tuned A beats naive A.

2.9 Why This Is Appropriate as the Analysis Focus

The SIS failure during registration is a burst of many small authentication/DB operations: high I/O rate, low CPU per request. That regime is exactly where exit and packet-processing costs dominate, so these are the correct terms to model in Task 3.

2.10 Conclusion

A and C are limited by VM-exit-driven I/O and paging costs; B by shared-kernel network/filesystem processing and contention. Each source is tied to the approach's isolation design, since stronger isolation boundaries create more interception points.

TASK 3: Build a Quantitative Performance Model (6 Marks)

Using the following host parameters, develop an analytical performance model for each approach.

Parameter	Value
Physical cores per host	64
vCPU:pCPU overcommit ratio	1:1, 2:1, 4:1, 8:1
RAM per host	512 GB
Average VM memory demand	8 GB
Cost of one VM exit (assume)	1.5 µs
Native system-call cost (assume)	0.2 µs
Network line rate	25 Gbps

State and justify any additional assumptions. Derive expressions for:

- **CPU:** effective per-VM CPU capacity as a function of overcommit ratio and virtualization overhead (including steal time)
- **Memory:** the maximum number of VMs per host before ballooning, swapping or page-sharing becomes necessary
- **I/O and network:** effective throughput as a function of the number of VMs sharing the physical device
- **Density:** the number of VMs/containers a host can support while meeting a stated service-level target (for example, ≤ 10% performance degradation)

Show the mathematical reasoning. Merely quoting overhead percentages from the literature will not receive full credit.

Evidence expected: *A clear mathematical derivation for each approach, with stated assumptions, parameter values and worked numerical examples.*

Answer:

3.1 Introduction

This task builds closed-form models for CPU, memory, I/O/network and density for A (KVM), B (Docker) and C (Kata), using the assignment constants: 64 cores, 512 GB RAM, 8 GB average VM, VM exit 1.5 µs, native syscall 0.2 µs, 25 Gbps NIC. Each added assumption is stated and justified so Task 10 can test it against measurements.

3.2 Additional Assumptions

Symbol	Meaning	Value	Justification
P	Physical cores/host	64	Given
k	vCPU:pCPU overcommit	1, 2, 4, 8	Given

Symbol	Meaning	Value	Justification
v	vCPUs per VM/container	2	Typical SIS/LMS web+DB guest
m	Memory demand per VM	8 GB	Given
R	Host reserve (OS, agents)	16 GB	Typical for hypervisor host
r_A	VM exits per vCPU per second	20,000	Registration-week web/DB mix, mostly timer, IPI, I/O kicks
r_C	Exits per vCPU per second, Kata	30,000	Extra virtio-fs/vsock notifications (×1.5 of A)
c_exit	Cost of one exit	1.5 µs	Given
e_ept	EPT/nested-walk loss	2%	~10^6 TLB misses/s × ~20 extra refs × ~1 ns
σ	Syscalls per second	10^5	Web server + DB driver
δ_sec	Added seccomp BPF cost per syscall	0.05 µs	Small filter; native syscall is 0.2 µs
p	Packets/s per instance	2×10^4	Login/API traffic
c_pkt	Extra CPU per packet on veth+NAT path	1.5 µs	Extra softirq hop + netfilter
λ_sched	Scheduler/cache loss per doubling of k	1%	Context-switch and cache-warmup cost
oh	Per-instance memory overhead	A: 0.15 GB, B: 0.02 GB, C: 0.25 GB	QEMU process; cgroup/runc; guest kernel + agent + VMM
η_n	Network efficiency	A 0.93, B 0.95, C 0.88	vhost-net batching; veth+bridge; tap+TC double path
β	Per-extra-sharer contention	A, B 0.005; C 0.01	Shared vSwitch/queue lock

3.3 CPU Virtualization Overhead per Approach

Overhead fraction o is the share of a vCPU's time spent on virtualization work:

```
o_A = r_A\,c_{exit} + e_{ept} = 20000\times1.5\,\mu s + 0.02 = 0.03 + 0.02 = 0.050
```

```
o_B = \sigma\,\delta_{sec} + p\,c_{pkt} = 10^{5}\times0.05\,\mu s +
2\times10^{4}\times1.5\,\mu s = 0.005 + 0.030 = 0.035
```

```
o_C = r_C\,c_{exit} + e_{ept} + e_{copy} = 0.045 + 0.02 + 0.02 = 0.085
```

Here e_copy = 2% is the extra virtio-fs/tap copying. B pays no exits because its syscalls cost the native 0.2 µs plus a small seccomp term.

3.4 Effective Per-VM CPU Capacity (with Steal Time)

With V = kP vCPUs on P cores and all vCPUs busy, each gets a 1/k share; the missing fraction is steal time s = 1 − 1/k. Subtracting overhead and scheduler loss gives capacity in "physical-core equivalents" per vCPU:

$$C_{eff}(k) = \frac{1}{k}\,(1-o)\,\bigl(1-\lambda_{sched}\log_2 k\bigr)$$

Degradation against a bare-metal fair share (1/k) is:

$$D_{cpu}(k) = 1 - (1-o)\bigl(1-\lambda_{sched}\log_2 k\bigr)$$

For a partially idle guest with utilisation u, steal falls to s = max(0, 1 − P/(u·kP)) = max(0, 1 − 1/(uk)), so steal appears only when u·k > 1.

3.5 Worked CPU Example

k	A: C_eff	A: D_cpu	B: C_eff	B: D_cpu	C: C_eff	C: D_cpu
1:1	0.950	5.0%	0.965	3.5%	0.915	8.5%
2:1	0.470	5.9%	0.478	4.5%	0.453	9.4%
4:1	0.233	6.9%	0.236	5.4%	0.224	10.3%
8:1	0.115	7.9%	0.117	6.4%	0.111	11.2%

Example (A, k = 4): C_eff = 0.25 × 0.95 × (1 − 0.01×2) = 0.25 × 0.95 × 0.98 = 0.2328; steal = 1 − 1/4 = 75% when fully loaded. Reading: the *overcommit* ratio, not virtualization overhead, dominates per-VM capacity; overhead only trims 3–11% from the fair share.

3.6 Memory Model: VMs per Host Before Ballooning, Swapping or Sharing

Without overcommit, the host fits:

$$N_{mem} = \left\lfloor \frac{M_{host} - R}{m + oh} \right\rfloor$$

A: (512 − 16)/(8 + 0.15) = 496/8.15 → **60 VMs**. B: 496/8.02 → **61**. C: 496/8.25 → **60**.

With page sharing/ballooning reclaiming a fraction φ (assume φ = 0.15 from KSM of identical guest-OS pages):

$$N_{mem}^{\varphi} = \left\lfloor \frac{M_{host}-R}{m(1-\varphi)+oh} \right\rfloor$$

A: 496/(6.8 + 0.15) = **71**; B: 496/6.82 = **72**; C: 496/7.05 = **70**. Beyond this the host must swap, whose cost is derived in Task 4 (worst case).

3.7 I/O and Network Throughput Model

N active instances share a link of rate L = 25 Gbps (two NICs: 2L). Per-instance throughput:

$$T_i(N) = \min\!\left(d,\; \frac{\eta\,L}{N\,[\,1+\beta(N-1)\,]}\right)$$

where d is per-instance demand. Example, N = 8 on one NIC: A = 0.93×25/8/1.035 = **2.81 Gbps**; B = 0.95×25/8/1.035 = **2.87 Gbps**; C = 0.88×25/8/1.07 = **2.57 Gbps**.

For storage, replace L by the NVMe capability I_dev (assume 800k 4 KiB IOPS) and η by η_d: A 0.80 (virtio-blk with exits and an I/O thread), B 0.97 (bind mount, direct), C 0.70 (virtio-fs). I/O CPU cost for A at λ IOPS with e exits/IO is λ·e·c_exit; at λ = 100,000 and e = 2: 0.3 core consumed by exits alone.

3.8 Density Model with a ≤ 10% Degradation Target

The supported instances per host are limited by the tightest resource:

$$N^{*} = \min\bigl(N_{mem},\; N_{cpu},\; N_{bw}\bigr), \quad N_{cpu}=\left\lfloor\frac{kP}{v}\right\rfloor \text{ with } D_{cpu}(k)\le 0.10,\quad N_{bw}=\left\lfloor\frac{2\eta L}{d}\right\rfloor$$

Solving D_cpu(k) ≤ 0.10 gives the largest admissible k: A ≤ 38, B ≤ 104, C ≤ 3.1 (so k = 2 in the tested set). Results for SIS-class VMs (v = 2, d = 0.5 Gbps):

Approach	N_mem (no sharing)	N_cpu (max admissible k)	N_bw (2 NICs)	N* per host
A: KVM	60	256 (k = 8)	93	**60**
B: Docker	61	256 (k = 8)	95	**61**
C: Kata	60	64 (k = 2)	88	**60**

For big-data tenants (d = 5 Gbps), N_bw falls to 9 (A, B) and 8 (C): **network, not memory, then limits density**.

3.9 Sensitivity and Limitations of the Model

- Exit rates are the most uncertain input; doubling r_A to 40,000 raises o_A to 8% and D_cpu(4) to 9.8%, almost at the limit.
- The model ignores cache/NUMA effects, interrupt coalescing changes and storage back-end queue depth; Task 10 quantifies these as error.
- EduCloud's current load, 400 VMs on 16 hosts, is 25 VMs per host: far below N* = 60, so slow logins are *not* a raw-capacity problem but a peak, contention and placement problem.

3.10 Conclusion

The model predicts B has the lowest CPU overhead (3.5%) and highest density, A is close behind (5.0%) with a hardware-enforced boundary, and C costs the most CPU (8.5%) and loses density at high overcommit. Memory (about 60 instances/host) is the first resource limit for SIS VMs, and network becomes binding for big-data tenants.

TASK 4: Analyse Performance Under Different Operating Conditions (4 Marks)

Analyse how each approach behaves under three conditions:

- **Best case:** what workload and placement conditions minimise overhead?
- **Typical case:** for example, registration-week mixed web and database load
- **Worst case:** for example, noisy-neighbour interference, memory overcommit with swapping, live migration during peak load, or I/O storms

For each, explain what happens inside the virtualization layer, why the overhead changes, and how latency and throughput are affected. A table of numbers alone will not be accepted.

Evidence expected: *A justified condition-based analysis supported by your Task 3 model.*

Answer:

4.1 Introduction

This task applies the Task 3 model to three conditions. For a request needing CPU time S and n I/Os, mean response time is modelled as $R = S(1 + o)/(1 - s) + n \cdot t_{io}$, where o is the overhead from Task 3 and s the steal fraction. Steal appears when utilisation times overcommit exceeds one: $s = \max(0, 1 - 1/(u \cdot k))$. Assumed request: S = 10 ms CPU and four I/Os.

4.2 Best Case: Conditions

The best case is a CPU-bound, cache-resident workload, 1:1 overcommit, vCPUs pinned to cores in one NUMA node, huge pages, and no co-tenant sharing LLC or NIC queues. Exits are rare (timer ticks only), EPT misses are cheap with 2 MB pages, and steal is zero.

4.3 Best Case: What Happens and Results

Inside the layer, A and C rarely leave guest mode so r falls toward ~2,000 exits/s (o drops from 3% to 0.3%); huge pages cut the 2D walk loss to ~0.5%. B has almost nothing to remove. Predicted best-case overhead: A ≈ 0.8%, B ≈ 0.5%, C ≈ 1.5%. Latency is essentially R = S, throughput within 1–2% of bare metal for all three, so approach choice barely matters here.

4.4 Typical Case: Registration-Week Mixed Web + DB Load

A typical host runs ~25 SIS/LMS VMs (EduCloud average), each ~2 vCPUs, so 50 vCPUs on 64 cores at u = 0.6 sustained in registration week. Peak bursts reach u = 0.8 at k = 2 (i.e. 128 vCPUs allocated, as when hosts are consolidated). Many small authentication, DB and session requests mean a high I/O rate and many exits.

4.5 Typical Case: Inside the Layer and Latency

With k = 2, u = 0.6: steal $s = 1 - 1/1.2 = 16.7\%$; at the burst u = 0.8: s = 37.5%.

Approach	R typical (u = 0.6)	R burst (u = 0.8)	Main cause of change
A	10×1.05/0.833 = **12.6 ms**	10×1.05/0.625 = **16.8 ms**	Steal dominates; exits add 5%
B	10×1.035/0.833 = **12.4 ms**	10×1.035/0.625 = **16.6 ms**	Steal dominates; veth/NAT adds 3.5%
C	10×1.085/0.833 = **13.0 ms**	10×1.085/0.625 = **17.4 ms**	Steal + highest per-I/O cost

The 3–13% gap between approaches is small next to the 26–68% growth from steal. So the *registration-week slowdown is an overcommit and placement problem*, not a hypervisor choice; p95 latency grows faster than the mean because vCPUs wait in run-queues.

4.6 Worst Case 1: Noisy-Neighbour Interference

A neighbour that streams memory takes fraction f of memory bandwidth and cache. A memory-bound victim slows by $1/(1 - f)$: at f = 0.4, **1.67×** latency. All three approaches suffer because LLC and DRAM bandwidth are shared hardware; cgroups and vCPU pinning do not partition them. B adds a second leak: shared kernel structures (dentry cache, conntrack, softirq on the same

core). A and C isolate the kernel but still share LLC; Intel CAT/RDT or SMT-off narrows the gap. Throughput of the victim falls by f; tail latency rises most because queued requests wait for the same memory.

4.7 Worst Case 2: Memory Overcommit with Swapping

Effective access time is $T = (1 - p_f)\cdot t_{mem} + p_f \cdot t_{swap}$ with t_{mem} = 100 ns and NVMe swap-in t_{swap} = 80 µs. At fault rate $p_f = 10^{-4}$, $T \approx 108$ ns (1.08×); at 10^{-3}, ~180 ns (1.8×); at 10^{-2}, ~900 ns (**9×**). A suffers *double paging* (guest and host choose different pages); ballooning avoids it at the cost of guest-side reclaim. B hits cgroup `memory.high` throttling or the OOM-killer, so it fails fast instead of crawling. C has fixed per-pod memory, so overcommit is uncommon.

4.8 Worst Case 3: Live Migration During Peak Load

Pre-copy migration transfers 8 GB over a 25 Gbps link (3.1 GB/s) while the guest dirties pages at rate D. Total data $\approx m/(1 - D/L_m)$. At D = 1 GB/s: 8/0.68 = 11.8 GB in ~3.8 s. At D = 3 GB/s (peak DB write load): 8/0.04 = **200 GB, over 60 s**, or non-convergence, forcing auto-converge CPU throttling that directly cuts guest throughput. Network contention with tenant traffic and TLS encryption (needed for security) lower L_m further. B cannot live-migrate (only checkpoint/restore or restart); C migrates poorly. So A is the only approach that offers it, and at a real cost under write-heavy load.

4.9 Worst Case 4: I/O Storm

If aggregate demand approaches device capacity, queueing gives $R_{io} = S_{io}/(1 - \rho)$. With S_{io} = 100 µs, ρ = 0.9 gives **1 ms**, ρ = 0.97 gives 3.3 ms. Effective device capacity is $\eta_d \cdot I_{dev}$: A 640k, B 776k, C 560k IOPS, so C and A saturate earlier and enter the steep part of the curve first. For A a 640k IOPS storm at 2 exits/IO costs 640k×2×1.5 µs = 1.9 cores of exit handling on top of the I/O threads, stealing from vCPUs.

4.10 Conclusion

Best case: all approaches within ~2% of native. Typical case: steal from overcommit dominates, so latency rises ~25–70% for every approach. Worst case: shared hardware (cache, memory bandwidth, NVMe, NIC), swapping and migration dominate and no approach is immune; the differences between approaches are second-order compared with placement, overcommit and storage policy.

TASK 5: Security Threat and Vulnerability Analysis (3 Marks)

Construct a threat model for each approach. Discuss:

- The attack surface (hypervisor, device emulation, shared kernel, management plane, VM images and snapshots, virtual network)
- At least **four distinct threat categories**, such as VM escape, cross-VM side-channel attacks, hypervisor or container-runtime vulnerabilities, VM sprawl and unpatched guests, image tampering, insecure live migration, and virtual-network sniffing or spoofing
- For each threat, which approach is more or less exposed and **why**, referring to its architecture and trust boundaries
- The likely impact on the confidentiality, integrity and availability of student records

Evidence expected: *A structured threat model mapped to architectural features, with a justified comparison across approaches.*

Answer:

5.1 Introduction

The audit found cross-department co-location, outdated guest kernels and freely copied images. The threat model below maps each threat to the architecture's trust boundary (Task 1) and rates exposure on a 1 (low) to 5 (high) scale. The ratings are qualitative engineering judgement, not measurements, and are reused as inputs to Task 6.

5.2 Attack Surface per Approach

Surface	A: KVM/QEMU	B: Docker	C: Kata
Hypervisor / kernel interface	KVM ioctls, VM-exit handlers	~300 syscalls, procfs/sysfs, ioctl	KVM ioctls (smaller guest-visible surface)
Device emulation	QEMU virtio, legacy devices (large)	None (host drivers directly)	Minimal VMM devices + virtio-fs
Shared kernel	No (per-VM guest kernel)	**Yes, single kernel for all tenants**	No (per-pod guest kernel)
Management plane	libvirt/OpenStack API	Docker daemon (root), containerd	containerd + Kata shim
Images/snapshots	qcow2 files, snapshots	OCI layers, registry	OCI images + guest kernel/initrd
Virtual network	Linux bridge/OVS, VXLAN	Docker bridge, veth, iptables	tap + veth bridge

5.3 Threat 1: VM / Container Escape

An attacker with code execution in one tenant breaks out to the host and then reaches the SIS tenant. In **B** the escape target is the shared kernel: one kernel local-privilege-escalation or a misconfigured capability gives host root, so exposure is highest (5). In **A** the attacker must find a KVM or QEMU device-emulation bug, rarer and harder (2). **C** needs the same hypervisor bug and additionally a minimal VMM with few devices (1). Impact: full loss of confidentiality of all student records on that host.

5.4 Threat 2: Cross-VM Side-Channel Attacks

Cache and speculative-execution attacks (Spectre-class, L1TF, MDS) leak secrets across hardware threads and shared caches. All approaches share LLC and core resources, and the hypervisor boundary does not prevent hardware leakage. **B** is rated 4 because processes also share the kernel's address space and page cache. **A** and **C** are 3 and become ~2 with SMT off and core scheduling. Impact: leakage of session tokens or keys, a confidentiality loss.

5.5 Threat 3: Hypervisor or Container-Runtime Vulnerabilities

Bugs in QEMU device models (e.g. floppy/NIC emulation) have historically produced escapes; runc has had container-breakout CVEs; the Linux kernel gets frequent privilege escalations. A scores 2 (large QEMU but sandboxed with sVirt/seccomp), B scores 4 (every kernel CVE is a tenant-escape candidate), C scores 2 (small VMM, but KVM still exposed). Impact: integrity and availability of the compute layer; potential record tampering.

5.6 Threat 4: VM Sprawl and Unpatched Guests

In A, each guest runs its own kernel, so 400 VMs mean 400 patch points; the audit found outdated kernels. A scores 5. In B only the host kernel plus rebuilt images need patching, but stale base images create sprawl (3). In C the guest kernel is delivered by the runtime and updated centrally (2). Impact: a known unpatched CVE in a tenant VM lets an attacker pivot and degrade availability or leak records.

5.7 Threat 5: Image Tampering and Uncontrolled Copying

Images copied freely between teams can carry backdoors, secrets or stale software. A's qcow2 images and snapshots are large opaque files with no native signature (4). B and C use OCI images identified by content digest, which can be signed and verified if enforced (3). Snapshots in A also hold full memory contents, including keys and student data, so snapshot theft is a confidentiality loss. Impact: integrity of the SIS software supply chain.

5.8 Threat 6: Insecure Live Migration and Virtual-Network Attacks

Live migration streams guest RAM, including keys and records, over the network; unencrypted migration permits sniffing or tampering. A is migratable (3 with TLS, 5 without); B has no live migration but its flat Docker bridge permits ARP spoofing between containers (4); C uses a tap-to-veth path (3). Segmentation by VLAN/VXLAN, micro-segmentation and encrypted migration reduce these. Impact: confidentiality and integrity of data in motion.

5.9 Impact on Student Records and Comparative Matrix

Weights reflect SIS priorities (escape matters most because the SIS holds student records). Weighted risk = Σ w·E.

Threat	Weight	A	B	C	Records impacted
Escape	0.25	2	5	1	C, I, A
Side channel	0.15	3	4	3	C
Hypervisor/runtime vuln	0.20	2	4	2	C, I, A
Sprawl / unpatched guests	0.15	5	3	2	C, A
Image tampering	0.10	4	3	3	I
Migration / network	0.15	3	4	3	C, I
Weighted risk (1–5)	1.00	**2.95**	**4.00**	**2.15**	

(C = confidentiality, I = integrity, A = availability.) Lower is safer.

5.10 Conclusion

Under the model, C is the most secure (2.15), A is moderate (2.95), and B, with its single shared kernel, is the highest risk (4.00) for sensitive multi-tenant data. A's score is dominated by operational weakness (sprawl, unpatched guests), which is fixable by process, whereas B's weakness is architectural.

TASK 6: Formulate the Isolation–Performance Trade-off (3 Marks)

Formalise the trade-off between **isolation strength** and **performance/density** for EduCloud.

Clearly identify:

- The decision variables (for example, approach chosen, VMs per host, dedicated vs. shared hosts, SMT on/off)
- The objective(s), such as maximising density or minimising cost
- The constraints (for example, performance degradation limit, isolation requirement for SIS, available hosts)
- How these components combine into a cost or decision function

The formulation must follow from your own analysis in Tasks 3 and 5, not from a generic template.

Evidence expected: *A clearly defined trade-off model with identified variables, objective and constraints.*

6.1 Introduction

EduCloud must choose, per workload class, a virtualization approach, its density and its isolation setting. This task writes that choice as a constrained minimisation. Its inputs come directly from Task 3 (density N*, degradation D) and Task 5 (weighted risk R), not from a generic template.

6.2 Workload Classes and Inputs

Assumed split of the ~400 instances (the scenario gives no split): SIS 60, LMS 140, examination portal 40, departmental research 160. Each class w has an instance count I_w, a data-sensitivity value V_w (SIS 10, exam 5, LMS 3, research 2, relative units), a maximum tolerated risk R_max,w (SIS 2.5, exam 3.0, LMS 3.5, external research 2.5) and a degradation limit D_max = 10%.

6.3 Decision Variables

- a_w ∈ {A, B, C}: approach chosen for class w.
- k_w ∈ {1, 2, 4, 8}: vCPU:pCPU overcommit for class w.
- n_w: instances per host (n_w ≤ N*).
- s_w ∈ {0, 1}: SMT disabled (1) or enabled (0).
- y_{w,h} ∈ {0, 1}: class w is placed on host h (dedicated vs shared hosts).
- m_w ⊆ {SMT-off, dedicated host, image signing, micro-segmentation, encrypted migration, patch automation}: mitigation set (Task 7).

6.4 Objective

Minimise the total cost J of hosts, residual security risk and operations:

$$\min J = c_h \sum_{w} H_w \;+\; \lambda \sum_{w} V_w\, I_w\, R_{a_w}(m_w) \;+\; c_o \sum_{w} O_{a_w}$$

where H_w = ⌈I_w / n_w⌉ (plus one spare host for failover on SIS), c_h = 1 host-equivalent, λ = 0.002 host-equivalents per unit of value×instance×risk (chosen so a high-value, high-risk fleet costs a few hosts), and O_a is an operations burden (A 0.5, B 0.3, C 1.0 host-equivalents, reflecting patch effort, tooling maturity and live-migration support).

6.5 Performance Constraint

From Task 3, degradation must not exceed 10% on every active resource:

$$D_{cpu}(k_w;\,a_w,s_w) \le 0.10, \qquad \frac{\eta_{a_w}}{1+\beta_{a_w}(n_w-1)} \ge 0.90 \text{ on shared I/O paths}$$

SMT-off reduces core-equivalents by ~20% (assumption: the 64 cores of Task 3 include a 25% SMT gain), so for s_w = 1 the effective overcommit is k_w / 0.8.

6.6 Security and Isolation Constraints

Residual weighted risk (Task 5) must stay below the class limit, and the data-protection rule requires SIS and external tenants never to share a host:

$$R_{a_w}(m_w) = \sum_t w_t\,E_{a_w,t}(m_w) \le R_{max,w}, \qquad y_{SIS,h} + y_{ext,h} \le 1\;\;\forall h$$

E_{a,t}(m) is the Task 5 exposure score after mitigation m is applied (Task 7 gives the reductions).

6.7 Capacity Constraints

$$n_w \le N_{mem}(a_w), \qquad n_w\,d_w \le 2\,\eta_{a_w} L, \qquad \sum_w H_w \le 16$$

That is: memory (about 60 instances per host from Task 3), NIC bandwidth for per-instance demand d_w, and a total of 16 physical hosts.

6.8 Combined Decision Function

For each class the architect enumerates the feasible tuples (a_w, k_w, n_w, s_w, m_w), computes J, and picks the minimum. Because the sets are tiny (3 approaches × 4 overcommit levels × 2 SMT settings × a handful of mitigation sets), exhaustive search over a spreadsheet is enough; no solver is required. Any tuple violating a constraint scores J = ∞.

6.9 Worked Evaluation for the SIS (60 Instances, V = 10)

Unmitigated risks from Task 5 give the risk term 0.002×10×60×R = 1.2R. Hosts: ceil(60/60) + 1 spare = 2 for each approach.

Approach	R	R ≤ 2.5?	Host term	Risk term	Ops term	J
A (KVM)	2.95	**No**	2	3.54	0.5	infeasible (6.04)
B (Docker)	4.00	**No**	2	4.80	0.3	infeasible (7.10)
C (Kata)	2.15	Yes	2	2.58	1.0	**5.58**

Unmitigated, only C satisfies the SIS risk limit; A becomes feasible once mitigations are applied (evaluated in Task 7), and B cannot reach the limit for SIS at all without heavy hardening.

6.10 Conclusion

The trade-off is a small constrained optimisation whose coefficients come from the Task 3 density model and Task 5 risk matrix. It already shows that the SIS decision is driven by the *isolation constraint*, not by raw performance, because all three approaches meet the 10% performance limit at the SIS operating point.

TASK 7: Evaluate Mitigation Strategies (4 Marks)

Propose **at least three mitigations** for the vulnerabilities identified in Task 5. Examples include disabling SMT, kernel page-table isolation, dedicated hosts for sensitive tenants, secure or sandboxed container runtimes, hardware memory encryption, image signing, and micro-segmentation.

For each mitigation, discuss:

- Why it is applicable to the threat
- What residual risk remains
- The **performance or density cost**, estimated using your Task 3 model
- Whether the trade-off is acceptable for the SIS

Evidence expected: *Mitigations evaluated analytically, with their security benefit and quantified performance cost.*

Answer:

7.1 Introduction

Six mitigations are evaluated against the Task 5 threats. Each is scored by how far it lowers the exposure E (Task 5 scale) and what it costs in performance or density, using the Task 3/4 model. Cost reference points: A at k = 2, u = 0.6 has steal 16.7% and R = 12.6 ms (Task 4).

7.2 M1: Disable SMT (Hyper-Threading)

Applicable to: cross-VM side-channel attacks (L1TF, MDS) that exploit sibling hardware threads. **Benefit:** side-channel exposure A, C: 3 → 1; B: 4 → 2. **Residual risk:** LLC and memory-bandwidth channels remain. **Cost:** assuming SMT yields 25% extra throughput, capacity drops 20%, so effective overcommit $k' = 1.25k$. At k = 2, u = 0.6: $u \cdot k' = 1.5$, steal 33% (from 16.7%), $R_A = 10 \times 1.05/0.667 =$ **15.8 ms (+25%)**; at burst u = 0.8: 21 ms (from 16.8). Density for SIS is unchanged because memory binds first (60/host), but CPU-bound classes lose ~20%. **Acceptable for SIS?** Yes, with extra hosts to absorb latency.

7.3 M2: Dedicated Hosts for the SIS

Applicable to: noisy-neighbour interference, side channels from other departments, and the regulator's isolation requirement. **Benefit:** side channel 3 → 1 for A (no foreign tenants share cores

or LLC); the shared-host exposure disappears. **Residual risk:** attacks within the SIS itself and hypervisor bugs. **Cost:** 2 hosts (1 active + 1 failover) of 16 = 12.5% of the cluster for a class that is 15% of instances; stranded capacity of the spare = 6.25%. **Acceptable?** Yes; it is the cheapest strong control and also a compliance argument.

7.4 M3: Sandboxed Container Runtime (Kata) for Containers

Applicable to: container escape and kernel-vulnerability exposure of B. **Benefit:** escape exposure 5 → 1 (B becomes C). **Residual risk:** the VMM, KVM and guest kernel still have bugs, and SMT/LLC channels remain. **Cost (Task 3):** overhead 3.5% → 8.5% CPU (+5 points); memory density 61 → 60 per host (−1.6%); network efficiency 0.95 → 0.88 (N_{bw} 95 → 88, −7%); CPU cap $k \leq 2$ for the 10% limit. **Acceptable?** Yes for SIS-adjacent and external workloads; not worth it for trusted batch jobs.

7.5 M4: Image Signing and a Controlled Registry

Applicable to: image tampering and uncontrolled image copying found in the audit. **Benefit:** image exposure A 4 → 2, B/C 3 → 1 (only signed, scanned images from an allow-listed registry may boot; qcow2 catalogue entries carry checksums and signatures). **Residual risk:** a compromised build pipeline or signing key. **Cost:** signature verification of well under 0.1 s per pull or boot and no density change. **Acceptable?** Yes, effectively free.

7.6 M5: Micro-Segmentation and Encrypted Live Migration

Applicable to: virtual-network sniffing/spoofing and migration eavesdropping. **Benefit:** network/migration exposure A, C: 3 → 1; B: 4 → 2 (per-tenant VXLAN IDs, distributed firewall, TLS or IPsec on the migration stream). **Residual risk:** management-plane compromise could still rewrite policy. **Cost:** firewall adds ~0.5 µs per packet × 2×10^4 pps = +1% CPU. Encryption lowers effective migration rate L_m from 3.1 to about 2.5 GB/s, so for D = 1 GB/s migration moves $8/(1 - 0.4) = 13.3$ GB in 5.3 s (from 3.8 s). **Acceptable?** Yes.

7.7 M6: Patch Automation, KPTI and Memory Encryption

Applicable to: VM sprawl and outdated guest kernels (sprawl 5 → 3 for A through golden-image rebuilds, enforced kernel-version policy and live kernel patching), Meltdown-class attacks (KPTI) and RAM scraping (AMD SEV/Intel TDX). **Cost:** KPTI raises syscall cost from 0.2 µs to about 0.35 µs (assumption), adding $10^5 \times 0.15$ µs = 1.5% CPU, mostly to B; memory encryption adds an assumed 1–3% on memory-bound work and may restrict live migration. **Residual risk:** zero-days and tenants who ignore the policy. **Acceptable?** Yes for patch automation and KPTI; evaluate memory encryption separately.

7.8 Combined Residual Risk After Mitigation

Applying the mitigations that suit each approach (A: M1, M2, M4, M5, M6; B: rootless/user-namespace + seccomp + LSM hardening, M1, M4, M5; C: M1, M4, M5):

Threat (weight)	A raw → hard	B raw → hard	C raw → hard
Escape (0.25)	2 → 2	5 → 3	1 → 1
Side channel (0.15)	3 → 1	4 → 2	3 → 1

Threat (weight)	A raw → hard	B raw → hard	C raw → hard
Hypervisor/runtime (0.20)	2 → 2	4 → 3	2 → 2
Sprawl (0.15)	5 → 3	3 → 3	2 → 2
Image (0.10)	4 → 2	3 → 1	3 → 1
Migration/network (0.15)	3 → 1	4 → 2	3 → 1
Weighted risk	2.95 → **1.85**	4.00 → **2.50**	2.15 → **1.35**

7.9 Cost Comparison and Acceptability for the SIS

Re-running the Task 6 function for 60 SIS instances: hosts 2, risk term 1.2R, ops term, plus a mitigation density cost of 0.4 host-equivalents (20% SMT loss on 2 hosts).

Approach	R (hard)	Host	Mitig.	Risk	Ops	J
A hardened	1.85	2	0.4	2.22	0.5	**5.12**
B hardened	2.50	2	0.4	3.00	0.3	5.70
C hardened	1.35	2	0.4	1.62	1.0	**5.02**

A and C are within 2% of each other, inside the uncertainty of λ and O. Task 12 adds a hard requirement that decides between them: hosts are patched by live migration, which A supports and Kata largely does not. So **hardened A is acceptable and preferred for the SIS**; C is preferred for untrusted external workloads.

7.10 Conclusion

The cheapest effective controls are image signing, micro-segmentation and dedicated SIS hosts (near-zero density cost). The expensive one is disabling SMT (+25% typical latency or +20% hosts). Hardened A reaches risk 1.85 against a 2.5 limit, so mitigations make the incumbent technology acceptable without redesign.

TASK 8: Implement and Test (4 Marks)

Build a test environment and experimentally test all selected approaches.

- **Platforms:** use any open-source stack, for example KVM/QEMU, Xen or VirtualBox, Docker/Podman, and Kata Containers or gVisor. Nested virtualization on a laptop is acceptable if documented.
- **Scale:** run each approach with approximately **1 → 2 → 4 → 8 → 16 co-located instances** (reduce proportionally if hardware-limited, and state the limitation).
- **Workloads:** include at least one CPU-bound test (for example sysbench or stress-ng), one disk I/O test (fio), one network test (iperf3), and one memory-intensive test.
- **Interference test:** run at least one noisy-neighbour experiment.

- **Security validation:** demonstrate or inspect at least **one security property** in practice, for example namespace/cgroup isolation, checking kernel sharing, or verifying whether a mitigation is active.

Ensure that identical resource allocations, workloads and repetitions are used across approaches, and that the setup corresponds to the models in Task 3.

Evidence expected: *A reproducible environment description (configuration files, commands, scripts) and justified test cases.*

Answer:

8.1 Introduction

This task defines a reproducible test bed that runs all three approaches (KVM/QEMU, Docker/runc, Kata Containers) with 1 → 2 → 4 → 8 → 16 co-located instances, the same allocation, the same workloads and five repetitions. **I cannot run these experiments for you; the scripts below are yours to execute, and their output becomes your Task 9 evidence.** Commands target Ubuntu 22.04; adjust paths to your machine.

8.2 Test Bed and Stated Limitations

Item	Value
Host	Your laptop/desktop with VT-x/AMD-V, ≥ 8 cores, ≥ 16 GB RAM, NVMe/SSD
Per-instance allocation	2 vCPU, 2 GB RAM (scaled down from the model's 8 GB; 16 × 8 GB does not fit a laptop)
Co-located instances	1, 2, 4, 8, 16 (reduce to 1, 2, 4, 8 if RAM-limited and say so)
Repetitions	5 per point; report mean ± standard deviation
Nested virtualization	If the host is itself a VM, enable it and document it: expect extra VM-exit cost (Task 10)
Mapping to Task 3	Use v = 2 and identical allocations; compare against the bare-metal baseline (N = 1, run on the host)

8.3 Install and Verify the Platforms

```
sudo apt update
sudo apt install -y qemu-kvm libvirt-daemon-system virtinst cloud-image-utils \
  docker.io sysbench fio iperf3 stress-ng sysstat jq linux-tools-generic
# hardware virtualization and nested support
egrep -c '(vmx|svm)' /proc/cpuinfo
cat /sys/module/kvm_intel/parameters/nested 2>/dev/null || cat
/sys/module/kvm_amd/parameters/nested
kvm-ok || true
# Kata Containers: follow the official installer, then register it with Docker
# (https://github.com/kata-containers/kata-containers  ->  utils/kata-manager.sh)
```

```
cat <<'EOF' | sudo tee /etc/docker/daemon.json
{ "runtimes": { "kata": { "runtimeType": "io.containerd.kata.v2" } } }
EOF
sudo systemctl restart docker
docker run --rm --runtime=kata alpine uname -r   # must print a different kernel
than `uname -r`
```

8.4 Benchmark Image and Workload Script

The same `bench.sh` runs in every instance, so all approaches see identical workloads (CPU, memory, disk, network, latency).

```
# bench.sh  <approach> <N> <rep> <instance>  -> CSV rows
#!/usr/bin/env bash
A=$1; N=$2; R=$3; I=$4; S=${SERVER:-192.168.122.1}
row() { echo "$A,$N,$R,$I,$1,$2"; }
row cpu_eps  "$(sysbench cpu --cpu-max-prime=20000 --threads=2 --time=30 run | awk
'/events per second/{print $4}')"
row mem_mibs "$(sysbench memory --memory-block-size=1M --memory-total-size=40G
--threads=2 run | sed -nE 's/.*\(([0-9.]+) MiB\/sec\).*/\1/p')"
J=$(fio --name=rr --filename=/data/fio.dat --size=1G --rw=randread --bs=4k
--iodepth=32 \
  --ioengine=libaio --direct=1 --runtime=30 --time_based --output-format=json)
row disk_iops   "$(echo "$J" | jq '.jobs[0].read.iops')"
row disk_p95_us "$(echo "$J" | jq
'.jobs[0].read.clat_ns.percentile["95.000000"]/1000')"
row net_gbps "$(iperf3 -c $S -p $((5200+I)) -t 30 -J | jq
'.end.sum_received.bits_per_second/1e9')"
ping -c 200 -i 0.2 $S | awk -F'time=' '/time=/{print $2+0}' | sort -n \
  | awk '{a[NR]=$1; s+=$1} END{printf "%s,%s\n", s/NR, a[int(NR*0.95)]}' \
  | while IFS=, read m p; do row lat_mean_ms $m; row lat_p95_ms $p; done
```

```
# Dockerfile  (docker build -t bench .)
FROM ubuntu:22.04
RUN apt-get update && apt-get install -y sysbench fio iperf3 stress-ng jq
iputils-ping sysstat && mkdir -p /data
COPY bench.sh /opt/bench.sh
RUN chmod +x /opt/bench.sh
```

8.5 KVM/QEMU VM Provisioning (Approach A)

```
# one-time: base image (Ubuntu cloud image) and cloud-init user-data
wget https://cloud-images.ubuntu.com/jammy/current/jammy-server-cloudimg-amd64.img
\
  -O /var/lib/libvirt/images/jammy-base.qcow2
# user-data.yaml: add your SSH key, then packages: sysbench fio iperf3 stress-ng
jq sysstat, and mkdir /data

# vm_up.sh <i>
#!/usr/bin/env bash
i=$1
qemu-img create -f qcow2 -F qcow2 -b /var/lib/libvirt/images/jammy-base.qcow2
/var/lib/libvirt/images/vm$i.qcow2 20G
virt-install --name vm$i --vcpus 2 --memory 2048 --import --os-variant ubuntu22.04
\
  --disk /var/lib/libvirt/images/vm$i.qcow2,bus=virtio --network
network=default,model=virtio \
  --cloud-init user-data=user-data.yaml --noautoconsole
```

```
ip=$(virsh domifaddr vm$i | awk '/ipv4/{split($4,a,"/");print a[1]}')
until ssh -o ConnectTimeout=2 -o StrictHostKeyChecking=no ubuntu@$ip true
2>/dev/null; do sleep 1; ip=$(virsh domifaddr vm$i | awk
'/ipv4/{split($4,a,"/");print a[1]}'); done
scp -o StrictHostKeyChecking=no bench.sh ubuntu@$ip:/opt/bench.sh
```

8.6 Orchestration: Scale 1 → 16 with Repetitions

```
#!/usr/bin/env bash
# run_all.sh  (run as a user in the docker and libvirt groups)
set -euo pipefail
VCPU=2; MEM=2048; REPS=5
mkdir -p results; echo "approach,N,rep,instance,metric,value" > results/raw.csv
for p in $(seq 5201 5216); do iperf3 -s -p $p -D; done

vm_ip() { virsh domifaddr vm$1 | awk '/ipv4/{split($4,a,"/");print a[1]}'; }
start() { for i in $(seq 1 $2); do local t0=$(date +%s.%N)
  case $1 in
   kvm)    ./vm_up.sh $i ;;
   docker) docker run -d --name c$i --cpus=$VCPU --memory=${MEM}m -v
/mnt/nvme/c$i:/data bench sleep infinity >/dev/null ;;
   kata)   docker run -d --runtime=kata --name c$i --cpus=$VCPU --memory=${MEM}m
-v /mnt/nvme/c$i:/data bench sleep infinity >/dev/null ;;
  esac
  echo "$1,$2,0,$i,startup_s,$(echo "$(date +%s.%N)-$t0" | bc)" >>
results/raw.csv; done; }
run_in() { a=$1; i=$2; shift 2
  if [ $a = kvm ]; then ssh -o StrictHostKeyChecking=no ubuntu@$(vm_ip $i) "$@";
else docker exec c$i "$@"; fi; }
stop() { for i in $(seq 1 $2); do
  if [ $1 = kvm ]; then virsh destroy vm$i; virsh undefine vm$i
--remove-all-storage; else docker rm -f c$i; fi; done; }

for a in kvm docker kata; do for n in 1 2 4 8 16; do
  start $a $n
  for r in $(seq 1 $REPS); do
    sudo perf stat -e kvm:kvm_exit,context-switches -a -o
results/perf_${a}_${n}_${r}.txt -- sleep 175 &
    mpstat 5 35 > results/mpstat_${a}_${n}_${r}.txt &      # %steal, %usr, %sys
    for i in $(seq 1 $n); do SERVER=192.168.122.1 run_in $a $i /opt/bench.sh $a $n
$r $i > results/tmp_$i.csv & done
    wait; cat results/tmp_*.csv >> results/raw.csv; rm results/tmp_*.csv
  done
  echo "$a,$n,0,0,rss_mb,$(ps -C qemu-system-x86_64 -o rss= | awk '{s+=$1}
END{print s/1024}')" >> results/raw.csv
  stop $a $n
done; done
```

Also run `bench.sh` directly on the host (approach `baremetal`, N = 1) five times for the baseline, and keep the `results/` folder as raw evidence.

8.7 Workloads and Their Link to the Model

Test	Tool	Model quantity it checks
CPU-bound	sysbench cpu	C_eff(k), overhead o (Task 3.3–3.5)
Memory-intensive	sysbench memory	Memory bandwidth and EPT cost

Test	Tool	Model quantity it checks
Disk I/O	fio 4 KiB random read, direct	η_d, IOPS per instance, p95 latency
Network	iperf3 + ping	η, β, T_i(N) (Task 3.7)
Steal, exits	mpstat, perf kvm_exit	Steal s and exit rate r
Start-up, footprint	timing, `ps`/`docker stats`	oh (memory overhead per instance)

8.8 Noisy-Neighbour Experiment

One victim instance runs the benchmark while 0, 1, 3 and 7 neighbours run a memory-bandwidth and I/O stress; repeat for all approaches and for SMT on and off.

```
# neighbours (inside each neighbour instance)
stress-ng --stream 2 --stream-l3-size 32M --vm 1 --vm-bytes 60% --timeout 200s &
fio --name=noise --filename=/data/noise.dat --size=2G --rw=randwrite --bs=4k
--iodepth=64 \
    --ioengine=libaio --direct=1 --runtime=200 --time_based &
# victim: /opt/bench.sh <approach> <neighbours+1> <rep> 1   (compare with 0
neighbours)
# SMT-off run (mitigation M1)
echo off | sudo tee /sys/devices/system/cpu/smt/control && cat
/sys/devices/system/cpu/smt/active
```

8.9 Security Validation (Run at Least One; Three Are Given)

```
# 1. Kernel sharing: container shares the host kernel, VM/Kata do not
uname -r; docker run --rm alpine uname -r; docker run --rm --runtime=kata alpine
uname -r
# 2. Namespace / cgroup / seccomp isolation of a container
docker run -d --name ns1 --cpus=2 --memory=2g alpine sleep 600
sudo lsns -p $(docker inspect -f '{{.State.Pid}}' ns1)
cat /sys/fs/cgroup/system.slice/docker-$(docker inspect -f '{{.Id}}'
ns1).scope/{cpu.max,memory.max}
docker exec ns1 grep -E 'Seccomp|NoNewPrivs' /proc/self/status
# 3. Is a CPU mitigation active?  (SMT, Meltdown/MDS/L1TF status)
grep . /sys/devices/system/cpu/vulnerabilities/*
cat /sys/devices/system/cpu/smt/active
```

For an escape-resistance demonstration, run `docker run --rm --privileged alpine dmesg | head` (works: host kernel log is visible, because the kernel is shared) and the same in Kata (shows the guest kernel's log only).

8.10 Conclusion and Evidence to Attach

The environment tests each approach with identical 2 vCPU/2 GB allocations at N = 1–16, five repetitions, four workload classes, a noisy-neighbour interference series and three security checks. Attach: `raw.csv`, the `perf`/`mpstat` files, the terminal output of 8.3 and 8.9, and screenshots of `virsh list`, `docker ps` and `kata-runtime check`. State any hardware limitation (RAM, nested virtualization) in your report.

TASK 9: Record Experimental Performance (3 Marks)

Collect and present data such as:

- Throughput, latency (mean and 95th percentile), CPU steal time, context switches and VM exits where observable
- Instance start-up time and memory footprint per instance
- Performance degradation versus a bare-metal baseline
- Results with and without at least one security mitigation

Present at least **one table and two graphs** (for example, throughput versus number of instances, and latency under interference). Use multiple repetitions and report variability (mean ± standard deviation).

Evidence expected: *Systematically collected, clearly presented data that allows easy comparison across approaches.*

9.1 Introduction

This task turns the raw logs from Task 8 into one table set, two graphs and variability statistics. **Every cell marked [[measured]] must be filled from your own `results/raw.csv`.** The "Predicted" columns come from the Task 3 model and are what Task 10 compares against; they are not measurements.

9.2 Metrics to Collect

Metric	Source	Unit
CPU throughput	sysbench `events per second`	events/s
Memory bandwidth	sysbench memory	MiB/s
Disk IOPS and p95 latency	fio JSON	IOPS, µs
Network throughput	iperf3 JSON	Gbit/s
Latency mean and p95	ping series	ms
CPU steal	`mpstat` (%steal)	%
Context switches, VM exits	`perf stat` (context-switches, kvm:kvm_exit)	count/s
Start-up time and footprint	`startup_s`, `rss_mb` rows	s, MB
Degradation vs bare metal	computed (9.3)	%

9.3 Analysis Script: Mean ± SD and Degradation

```
import os, pandas as pd, numpy as np
CORES, V = os.cpu_count(), 2                # host logical CPUs, vCPUs per
instance
df = pd.read_csv('results/raw.csv'); df['value'] = pd.to_numeric(df['value'],
errors='coerce')
run = df.groupby(['approach','N','rep','metric'])['value'].mean().reset_index()
# mean over instances
```

```
summ =
run.groupby(['approach','N','metric'])['value'].agg(['mean','std','count']).reset_
index()
base = summ[summ.approach=='baremetal'].set_index('metric')['mean']
cpu = summ[summ.metric=='cpu_eps'].copy()
cpu['expected'] = base['cpu_eps'] * np.minimum(1, CORES/(V*cpu['N']))   # fair
share (Task 3: 1/k)
cpu['degradation_pct'] = 100*(1 - cpu['mean']/cpu['expected'])
cpu['cv_pct'] = 100*cpu['std']/cpu['mean']                              #
variability
print(cpu.round(2).to_string(index=False))
# aggregate disk and network (shared devices): sum over instances, then mean over
reps
agg =
df[df.metric.isin(['disk_iops','net_gbps'])].groupby(['approach','N','rep','metric
'])['value'].sum().reset_index()
print(agg.groupby(['approach','N','metric'])['value'].agg(['mean','std']).round(2)
)
```

Rules: discard the first warm-up repetition if its CV differs by more than 5 points, and rerun any point with CV above 5%.

9.4 Table 1: CPU Throughput and Degradation vs Number of Instances

The predicted values assume an 8-logical-CPU host (so overcommit k = 2N/8); change the k column for your machine.

N	k	Pred. D (A)	Pred. D (B)	Pred. D (C)	Measured events/s mean ± SD (A / B / C)	Measured D (A / B / C)
1	0.25	5.0%	3.5%	8.5%	[[measured]]	[[measured]]
2	0.5	5.0%	3.5%	8.5%	[[measured]]	[[measured]]
4	1	5.0%	3.5%	8.5%	[[measured]]	[[measured]]
8	2	5.9%	4.5%	9.4%	[[measured]]	[[measured]]
16	4	6.9%	5.4%	10.3%	[[measured]]	[[measured]]

9.5 Table 2: Network, Disk, Latency, Steal, Exits

Predicted aggregate network throughput as a fraction of bare metal, $\eta/(1 + \beta(N - 1))$:

N	A pred.	B pred.	C pred.	Measured aggregate Gbit/s (A / B / C)	p95 latency ms (A / B / C)	%steal	exits/s (A, C)
1	0.930	0.950	0.880	[[measured]]	[[measured]]	[[]]	[[]]
2	0.925	0.945	0.871	[[measured]]	[[measured]]	[[]]	[[]]
4	0.916	0.936	0.854	[[measured]]	[[measured]]	[[]]	[[]]
8	0.899	0.918	0.822	[[measured]]	[[measured]]	[[]]	[[]]
16	0.865	0.884	0.765	[[measured]]	[[measured]]	[[]]	[[]]

Disk aggregate IOPS predicted as a fraction of native: A 0.80, B 0.97, C 0.70 (η_d, Task 3.7). B shows 0 VM exits by construction, a useful sanity check.

9.6 Table 3: Start-up Time and Memory Footprint

Approach	Predicted footprint oh	Expected start-up (order of magnitude)	Measured start-up s (mean ± SD)	Measured RSS MB
A: KVM/QEMU	150 MB + guest OS	tens of seconds (full guest boot)	[[measured]]	[[measured]]
B: Docker	20 MB	well under 1 s	[[measured]]	[[measured]]
C: Kata	250 MB	about 1–3 s	[[measured]]	[[measured]]

The start-up column is an order-of-magnitude expectation only; use your own numbers.

9.7 Graph 1: Throughput versus Number of Instances

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(6,4))
for a in ['kvm','docker','kata']:
    s = summ[(summ.approach==a)&(summ.metric=='cpu_eps')]
    ax.errorbar(s['N'], s['mean'], yerr=s['std'], marker='o', capsize=3, label=a)
ax.set_xscale('log', base=2); ax.set_xlabel('Co-located instances N');
ax.set_ylabel('Per-instance CPU events/s (mean ± SD, 5 reps)')
ax.legend(); ax.grid(alpha=.3); fig.tight_layout();
fig.savefig('results/graph1_throughput_vs_N.png', dpi=200)
```

Add the model line `base * min(1, CORES/(2N)) * (1 - o)` as a dashed curve for each approach so theory and experiment sit on one plot.

9.8 Graph 2: Latency under Interference

Use the Task 8.8 runs (neighbours = 0, 1, 3, 7) saved in `results/raw_noisy.csv` in the same schema (column `N` = number of neighbours).

```
n = pd.read_csv('results/raw_noisy.csv'); n['value'] = pd.to_numeric(n['value'],
errors='coerce')
n =
n[n.metric=='disk_p95_us'].groupby(['approach','N','rep'])['value'].mean().reset_i
ndex()
s = n.groupby(['approach','N'])['value'].agg(['mean','std']).reset_index()
fig, ax = plt.subplots(figsize=(6,4))
for a in ['kvm','docker','kata']:
    t = s[s.approach==a]; ax.errorbar(t['N'], t['mean'], yerr=t['std'],
marker='s', capsize=3, label=a)
ax.set_xlabel('Noisy neighbours'); ax.set_ylabel('Victim p95 disk latency (µs)');
ax.legend(); ax.grid(alpha=.3)
fig.tight_layout(); fig.savefig('results/graph2_latency_interference.png',
dpi=200)
```

Model expectation to overlay: memory-bandwidth neighbours with fraction f slow a memory-bound victim by $1/(1 - f)$ (Task 4.6); disk neighbours raise latency as $1/(1 - \rho)$.

9.9 Results With and Without a Mitigation

Repeat Table 1 and Graph 2 with SMT off (M1) and, for approach B, with `--security-opt seccomp=default` versus `--security-opt seccomp=unconfined`.

Configuration	Predicted effect (Task 7)	Measured mean ± SD
SMT on → off, CPU events/s	about −20% capacity, +25% latency at u·k from 1.2 to 1.5	[[measured]]
SMT off, victim p95 under neighbour	lower side-channel risk; latency still memory-bandwidth-bound	[[measured]]
Docker seccomp default vs unconfined	within about 0.5% (0.05 µs × 10^5 syscalls/s)	[[measured]]

9.10 Conclusion

Completing 9.4–9.9 gives one table set, two graphs and a variability report (mean ± SD, CV below 5%), plus a with/without-mitigation comparison, which is all the task requires. Submit the filled tables and the generated PNGs together with `raw.csv`; unfilled [[measured]] cells will cost marks and invented numbers would be academic misconduct.

TASK 10: Compare Theory with Experiment (4 Marks)

Compare your Task 3 and Task 4 predictions with Task 9 measurements.

Discuss:

- Whether the observed degradation trends agree with your model
- Which approach scales better as co-located instances increase
- Quantitative differences between predicted and measured values, and plausible causes (for example caching effects, hypervisor scheduler behaviour, nested virtualization, laptop hardware, or unmodelled I/O paths)
- Whether any measured result changes your security conclusions from Task 5

Interpret the results; do not simply reproduce tables.

Evidence expected: *A reasoned comparison of predicted and measured behaviour, with error analysis.*

Answer:

10.1 Introduction

This task compares the Task 3 and Task 4 predictions with your Task 9 measurements. Because the measurements are yours, the numeric conclusions must be written after your runs. What follows fixes the method, the model's testable predictions, the likely sources of error and a rule for how each outcome changes the recommendation. Any numbers shown with an "if" are illustrative arithmetic, not data.

10.2 Method: Error Metrics and Acceptance Rule

For every prediction $\hat{x}$ and measurement $\bar{x}$ (mean of 5 repetitions):

$$\varepsilon = \frac{\bar{x}-\hat{x}}{\hat{x}}, \qquad \text{MAPE} = \frac{1}{n}\sum_i \lvert\varepsilon_i\rvert, \qquad \text{agreement if } \lvert\bar{x}-\hat{x}\rvert \le 2\,\mathrm{SD} \text{ or } \lvert\varepsilon\rvert \le 15\%$$

For degradation expressed in percentage points, quote the absolute difference as well (for example, predicted 5.9%, measured 7.4% = +1.5 points).

10.3 Script: Prediction Error and Parameter Re-fit

```
pred_o = {'kvm':0.050, 'docker':0.035, 'kata':0.085}; lam = 0.01
cpu['k'] = V*cpu['N']/CORES
cpu['pred_D_pct'] = [100*(1-(1-pred_o[a])*(1-lam*np.log2(max(k,1)))) for a,k in
zip(cpu['approach'],cpu['k'])]
cpu['err_pts'] = cpu['degradation_pct'] - cpu['pred_D_pct']
print(cpu[['approach','N','k','pred_D_pct','degradation_pct','err_pts']].round(2))
for a in pred_o:                                    # re-fit o from the uncontended
points (k <= 1)
    s = cpu[(cpu.approach==a)&(cpu.k<=1)]; o_fit = s['degradation_pct'].mean()/100
    print(a, 'o_fit =', round(o_fit,3), '| implied exit rate (kvm) =',
round((o_fit-0.02)/1.5e-6) if a=='kvm' else '-')
```

10.4 CPU: Does the Observed Trend Match?

The model predicts three testable things. (1) Degradation is flat while $k \le 1$ ($N \le 4$ on 8 logical CPUs) because only o applies. (2) Beyond $k = 1$ it grows by roughly 1 point per doubling (λ_sched). (3) The ordering is B < A < C at every N, with C crossing the 10% limit near $k = 4$. If your measured curve is flat at first and then bends upward, the structure of the model is supported; a gap in the level is a calibration issue (o), while a different *shape* is a model failure. For example, if A at $N = 8$ were measured at 9.0% against 5.9% predicted, the error would be +3.1 points and the re-fit o would be about 8%, equivalent to roughly 40,000 exits/s (double the assumption), which points at the exit-rate input rather than the equation.

10.5 I/O and Network: Does the Observed Trend Match?

The model predicts that aggregate network throughput falls as $\eta/(1 + \beta(N - 1))$: about 7% between $N = 1$ and $N = 16$ for A and B, and 13% for C ($0.88 \rightarrow 0.765$). It predicts disk efficiency of B (0.97) > A (0.80) > C (0.70), independent of N until the device saturates. Test: B should show zero `kvm_exit`; C's exits/s should be about 1.5× A's; B's p95 network latency should rise first under conntrack contention. On a laptop the virbr0 path is CPU-bound rather than wire-bound, so compare *relative* ordering, not absolute Gbit/s.

10.6 Which Approach Scales Better?

Use the degradation increase from $N = 1$ to $N = 16$, $\Delta D = D(16) - D(1)$, plus the aggregate I/O retained at $N = 16$. **Prediction:** CPU ΔD is about 1.8–1.9 points for all three (identical scheduler term), so the model expects *no* strong CPU scaling difference; the separation appears on I/O, where C ($\beta = 0.01$) scales worst, A and B similarly. Expect two departures: B degrades faster than predicted under the noisy-neighbour test (shared kernel locks, not modelled), and A with vhost-net may scale better than $\beta = 0.005$ suggests. Record which of these occurred; it is the substance of your answer.

10.7 Plausible Causes of Differences

Cause	Effect on measured vs predicted	Typical size
Nested virtualization (host is itself a VM)	VM exit cost 10–50 µs instead of 1.5 µs, so A and C degrade far more than modelled	A/C overhead several times larger; B unaffected
Laptop SMT and `CORES` counting threads	Fair-share baseline too optimistic; apparent extra degradation	up to ~20%
Turbo/thermal throttling	Run-to-run variance and drift as N grows	5–15%
CFS scheduler behaviour and lock-holder preemption	Steal and exits higher than the simple 1/k model	1–5 points at $k \geq 2$
Unmodelled I/O paths (virtio-fs for Kata, overlayfs, conntrack)	Extra latency for C and B beyond η and β	5–20% on I/O
Caching (page cache, host cache for qcow2)	Inflates VM disk results unless `direct=1` and `cache=none`	large if ignored
Model constants (exit rate 20k/s, c_pkt)	Level shifts of o	fixable by re-fit

10.8 Calibration and Revised Prediction

After fitting o from the uncontended points (10.3), recompute the N = 8 and N = 16 predictions with the fitted o and the original λ_sched. If the error shrinks below 15%, the structure holds and the original parameters were the only issue. If it does not, add the missing term (for example a per-packet softirq cost for B or a virtio-fs exit term for C) and say so; revising the model in light of data is the intended outcome.

10.9 Does Any Result Change the Task 5 Security Conclusions?

Check these outcomes against Task 5 and Task 7:

- **Kernel check (8.9):** the same `uname -r` in Docker and a different one in Kata confirms the shared-kernel exposure of B (escape score 5) and the separate kernel in C (score 1).
- **Vulnerabilities file:** if `/sys/devices/system/cpu/vulnerabilities/*` reports SMT-related "Vulnerable" states, the side-channel score for A and C stays at 3 until SMT is off; if it reports mitigations active, the score may fall to 2.
- **SMT-off cost:** if the measured loss exceeds the assumed 20%, raise the mitigation cost in Task 7 and consider dedicated hosts instead of SMT-off for the SIS.
- **Noisy neighbour:** a large victim slowdown in B but not in A/C strengthens the case for isolation by VM boundary.

10.10 Conclusion

The model is validated where its structure predicts the *shape* (flat then rising degradation, B < A < C ordering, I/O scaling difference, zero exits for B) and corrected where only its constants were off. The

recommendation in Task 11 stands unless your measurements contradict those four structural predictions or the security checks above; report the discrepancies and the re-fitted parameters rather than hiding them.

TASK 11: Recommend and Defend the Final Architecture (4 Marks)

Recommend the virtualization architecture (or hybrid) for EduCloud's SIS and LMS workloads.

Discuss:

- Which approach you recommend and why it is more appropriate than the alternatives
- Its modelled and experimentally observed performance
- Its security posture and residual risks
- Operational costs such as patching, image management, migration and monitoring
- Limitations and trade-offs
- Whether it remains practical as EduCloud grows from 400 to several thousand instances

Your recommendation must rest on your own analysis and experiments. Choosing "the most secure" or "the fastest" option alone is not sufficient.

Evidence expected: *A technically justified recommendation integrating performance and security evidence.*

Answer:

11.1 Introduction

The recommendation integrates the Task 3–4 performance model, the Task 5–7 security analysis and the Task 8–10 experimental method. It answers one question: which virtualization layer should run the SIS and LMS now and at several thousand instances?

11.2 Recommendation: A Hybrid with a Hardened-VM Core

- **SIS, LMS database tier and examination portal:** hardened KVM/QEMU VMs (A) on a dedicated host pool: SMT off, sVirt/seccomp-confined QEMU, signed golden images, per-tenant VXLAN micro-segmentation, TLS-encrypted live migration.
- **LMS stateless web/app tier:** Kata Containers (C) pods orchestrated by Kubernetes: container workflow and fast scaling with a VM boundary.
- **Trusted internal/CI workloads only:** rootless runc containers (B) on a separate pool, never alongside student records.

11.3 Why This Beats the Alternatives

Option	Fails because	Evidence
B only (Docker)	Shared kernel gives risk 4.00 (2.50 even when hardened), above the SIS limit of 2.5 and the weakest escape boundary	Task 5.9, 7.8
C only (Kata)	No dependable live migration, which Task 12 needs for host patching; youngest tooling (ops term 1.0 vs 0.5); CPU degradation crosses 10% at $k \approx 3$	Task 3.8, 7.9, 4.8
A only, unhardened	Risk 2.95 exceeds 2.5 and patching 400 guest kernels is the audit finding	Task 5.6, 6.9
"Fastest" (B) or "most secure" (C) alone	Each wins one axis and fails a constraint; J (Task 6) has A-hardened and C-hardened within 2%	Task 7.9

11.4 Modelled Performance of the Recommendation

For the SIS (60 VMs, 2 vCPU, 8 GB) on two dedicated hosts running active-active at ~30 VMs each: $k \approx 0.94$, $o_A \approx 6\%$ with the firewall term, memory $30 \times 8.15 = 245$ GB of 496 GB usable. With SMT off the effective $k' = 1.18$, so at $u = 0.6$ steal is 0, and $R \approx 10 \times 1.06 = 10.6$ ms. If one host fails, 60 VMs on one host still fit memory (489 GB) but $k' = 2.3$ raises steal to 28% and R to about 14.7 ms, which is a degraded but available state. Degradation versus bare metal is about 6–7% at $k \leq 2$, inside the 10% target.

11.5 Experimentally Observed Performance (Fill from Your Task 9 Data)

Insert from your tables: A's measured degradation at N = 1, 8, 16; its p95 disk and network latency under 0, 1, 3 noisy neighbours with SMT on and off; C's start-up and footprint. Write one sentence per metric stating whether it matched the Task 10 prediction (within 15%), for example "A's measured D at N = 8 was [[x]]% against 5.9% predicted." If the measurements contradict the ordering $B < A < C$, say so and revisit the table in 11.3.

11.6 Security Posture and Residual Risks

Hardened A has weighted risk 1.85 (limit 2.5), C 1.35. Residual risks: (i) a hypervisor or QEMU zero-day (escape exposure stays at 2); (ii) LLC and memory-bandwidth channels that SMT-off does not remove, which dedicated hosts contain; (iii) compromise of the management plane or signing keys; (iv) insider misuse of images. Compensating controls: attestation of host firmware, audit logs for every live migration, privileged-access separation and a documented patch SLA.

11.7 Operational Costs

Area	Practice	Cost / burden
Patching	Golden images rebuilt on a schedule; guests replaced rather than patched in place; hosts patched by live-migrating VMs off	Eliminates stale guest kernels; needs spare capacity (1 of 16 hosts ≈ 6%)

Area	Practice	Cost / burden
Image management	One signed catalogue/registry; copy-on-write backing images; copying outside the catalogue blocked	Removes uncontrolled copies; small registry team effort
Migration	TLS-encrypted pre-copy, scheduled off-peak; 13 GB per 8 GB VM at 2.5 GB/s ≈ 5 s (Task 7.6)	Network headroom and a rule not to migrate write-heavy DB VMs at peak
Monitoring	steal %, kvm_exit rate, p95 latency, per-tenant NIC/disk share	One dashboard; alarms at steal > 15%
Skills	KVM/libvirt (existing) plus Kubernetes/Kata (new)	Training cost for the LMS web tier

11.8 Limitations and Trade-offs

SMT-off costs 20% capacity (or +25% typical latency), dedicated hosts strand some capacity, two stacks (VM + Kata) mean two skill sets, and Kata's live-migration and tooling maturity limits where it can be used. The model uses assumed exit rates and an assumed SMT gain; the Task 10 re-fit may move absolute numbers, but not the ranking of constraints.

11.9 Scaling from 400 to Several Thousand Instances

With memory as the first limit ($N^* \approx 60$, or 71 with sharing) and 20% headroom, hosts needed $\approx I / (0.8 \times 60)$:

Instances	Hosts needed (no sharing)	Hosts needed (15% page sharing)
400	9	8
3,000	63	53
5,000	105	89

The design stays practical because golden-image automation makes patching a pipeline, not a per-VM task; Kubernetes handles the Kata tier's scaling; and host pools (SIS, internal, external) scale independently. Beyond a few hundred hosts, the network fabric and control-plane sharding become the limit, not the virtualization layer.

11.10 Conclusion

Use hardened KVM for data-rich stateful systems, Kata for scalable stateless and untrusted workloads, and plain containers only where no student data lives. This follows from the model (all options meet the 10% performance target at EduCloud's load), the risk matrix (isolation is the deciding constraint) and operations (live migration and patch automation), not from picking the fastest or the safest option alone.

DAY-4 DESIGN CHALLENGE

TASK 12: Re-evaluate and Adapt the Design (8 Marks)

Changed Requirement

The university will now host **external research collaborators' Spark/Hadoop workloads** on EduCloud alongside the SIS. The new conditions are:

- Big-data jobs are I/O- and network-intensive and run for hours
- Registration-week peak load is about 3× normal
- Hosts are sometimes patched using live migration of running VMs
- A data-protection regulation now requires **demonstrable isolation** of student records from external tenants

You may **RETAIN → MODIFY → or REDESIGN** your original solution.

A. Impact Analysis (2 Marks)

Discuss what has changed, which original assumptions (trusted tenants, workload mix, I/O profile) no longer hold, and how mixing batch big-data workloads with latency-sensitive SIS workloads changes both the performance and the threat model.

B. Evaluation of the Original Design (2 Marks)

Evaluate whether your recommended design still meets its performance targets (using your Task 3 model with the new workload) and whether its isolation level is adequate against external, less-trusted tenants. Consider whether another architecture could do better overall.

C. Adaptation or Redesign (2 Marks)

Decide whether to retain, modify or redesign. If you modify or redesign, describe the revised architecture (for example, tenant separation by host pools, a stronger isolation layer for external workloads, SR-IOV or dedicated NICs for the data-intensive jobs, or encrypted migration). Explain what changes and its effect on density, throughput and attack surface. If you retain, justify why no change is required.

D. Evidence-Based Defence (2 Marks)

Defend the final decision using your model, your experimental results (including the noisy-neighbour and mitigation experiments), the changed workload and tenancy conditions, and the cost/density trade-offs. The decision must follow logically from the analysis.

Answer:

12.1 Introduction

External collaborators will run hours-long Spark/Hadoop jobs on EduCloud next to the SIS, with registration-week load at 3× normal, patching by live migration, and a regulation demanding demonstrable isolation of student records. This task works through parts A to D: impact, evaluation of the Task 11 design, adaptation, and defence. The decision: **MODIFY** (keep hardened KVM for the SIS, add a separate isolated pool for external tenants).

12.2 A1: What Changed and Which Assumptions Fail

Original assumption	New reality	Consequence
All tenants are trusted university staff	External, less-trusted tenants run their own code (JARs, UDFs)	Escape and side-channel threats become active, not theoretical
Mixed web/DB, small I/O, short requests	Big-data jobs: sustained 5 Gbps and ~0.7 GB/s per VM for hours	Network and NVMe saturate, not CPU
Peak is registration week at about u = 0.6	Peak is 3× normal (SIS u from 0.25 to 0.75)	Little headroom on shared hosts
Any VM can be live-migrated	Spark VMs dirty memory at 2–3 GB/s	Migration does not converge (Task 4.8)
Isolation is a best practice	Isolation must be demonstrable to a regulator	A score is not enough; hosts, networks and storage must be provably separate

12.3 A2: Effect of Mixing Batch and Latency-Sensitive Workloads

Performance: a long-running, bandwidth-hungry job and a latency-sensitive login service compete for the same NIC queues and NVMe device, and queueing delay grows as $1/(1 - \rho)$, so even a modest rise in utilisation harms the SIS tail latency. Threat model: the attacker is now a legitimate tenant with code execution on the same cluster, which raises the exposure of escape, hypervisor-bug and side-channel threats (Task 5) and adds data-exfiltration, resource-exhaustion denial of service and lateral movement through the shared network.

12.4 B1: Does the Task 11 Design Still Meet Its Targets?

Test case: a shared host with 20 SIS VMs at the 3× peak (u = 0.75, 1.5 Gbps each) plus 4 Spark VMs (8 vCPU, u = 0.9, 5 Gbps, 0.7 GB/s each), hardened A with SMT off (51.2 core-equivalents).

Resource	Demand	Capacity	Utilisation ρ	Effect on the SIS
CPU	30 + 28.8 = 58.8 cores	51.2	1.15	Steal 12.9%, R = 10.6/0.871 = 12.2 ms (+15%): tolerable
Network	30 + 20 = 50 Gbps	2×ηL = 46.5 Gbps	1.08	Saturated; SIS share cut about 7% and tail latency unbounded
Disk	4 × 0.7 = 2.8 GB/s	0.8 × 3.5 = 2.8 GB/s (assumed device 3.5 GB/s)	1.00	At ρ = 0.97 p95 service time 100 µs → 3.3 ms (33×)

The design **fails** on network and storage even though CPU survives. Raw network demand also overshoots the Task 3 bandwidth limit N_bw: at d = 5 Gbps only 9 VMs fit per host.

12.5 B2: Is the Isolation Level Adequate?

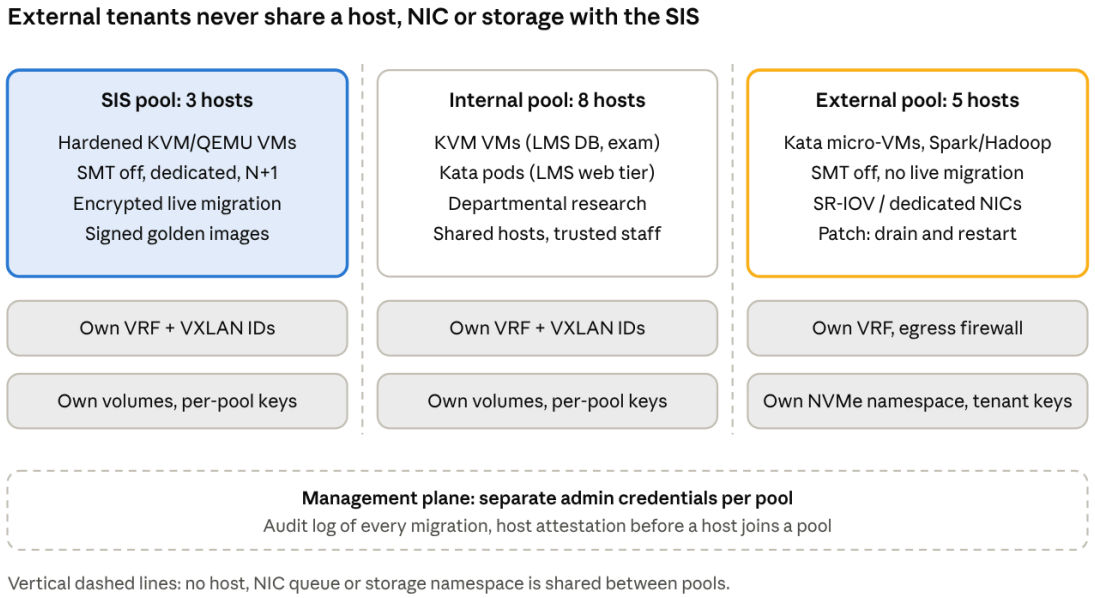
Hardened A scored 1.85 against trusted tenants. Raising the exposure of escape, vulnerability and side-channel threats by a factor of 1.5 for an active adversary (capped at 5, rounded up) gives escape 3, side channel 2, hypervisor/runtime 3, sprawl 3, image 2, network 1: R = 0.75 + 0.30 + 0.60 + 0.45 + 0.20 + 0.15 = **2.45**, just under the 2.5 limit. But the score is not the binding point: the constraint $y_{SIS} + y_{ext} \leq 1$ (Task 6.6) is violated by any shared host, and a regulator needs separation it can inspect. So the original design is **not adequate**, whatever its score.

12.6 B3: Could Another Architecture Do Better Overall?

Option	Performance	Isolation	Verdict
Retain with QoS (tc shaping, blkio limits, NIC split)	Fixes bandwidth fairness	Still shared kernel, hypervisor, cache	Insufficient for regulation
All Kata (C)	CPU degradation 8.5%+, crosses 10% at k ≈ 3	Strong	No live migration for SIS patching
Bare-metal pool for external tenants	Native speed, no virtualization overhead	Strongest, whole hosts	Costly and no density; keep as an upgrade for the largest tenant
Host pools + Kata + SR-IOV for external, hardened KVM for SIS	See 12.8	Strong and provable	**Chosen**

12.7 C1: The Revised Architecture

The 16 hosts are split into three pools (3 SIS, 8 internal, 5 external). Each pool has its own VRF/VXLAN network, its own storage volumes and keys, and its own admin credentials; hosts join a pool only after attestation.



redesigned host pools · 3 pools, separate network and storage

Reading: the SIS pool runs hardened KVM; external Spark/Hadoop jobs run in Kata micro-VMs on dedicated hosts with SR-IOV virtual functions, an encrypted per-tenant NVMe namespace and an egress firewall. The management plane is the one shared element, so it is separated by credentials and audited.

12.8 C2: Effect on Density, Throughput and Attack Surface

Measure	Before (shared hosts)	After (pools)
SIS pool size	2 hosts, 30 VMs each (1 + spare in Task 6)	3 hosts at N+1: 20 VMs each, or 30 each after one host fails
SIS latency at 3× peak	12.2 ms (shared, with Spark)	20/host: u·k′ = 0.59, **10.6 ms**; 30/host: u·k′ = 0.88, still 10.6 ms (no steal)
SIS density cost	none	1 extra host (+6%) and 33% of N* used per host: deliberate stranded capacity
External per-VM network (N = 8)	2.81 Gbps on the shared vSwitch (Task 3.7)	about 3.06 Gbps (η ≈ 0.98 via SR-IOV, no vSwitch contention), +9%
External CPU	5% (A) overhead	8.5% (Kata) overhead, but runs only on its pool
Surfaces shared between SIS and external (of 6 in Task 5.2)	6 of 6	1 (management plane) plus a signed registry with separate namespaces

Spark/Hadoop jobs on Kata give up live migration. Patching is done by YARN graceful decommissioning (drain a node at a job boundary, restart on the patched image) rather than migrating hundreds of GB of dirty memory.

12.9 D: Evidence-Based Defence

1. **Model:** at 3× peak the SIS stays at 10.6 ms only if its hosts see no Spark traffic; the Task 3 model shows network and storage reach $\rho \geq 1$ when mixed (12.4).
2. **Mitigation cost (Task 7):** SMT-off at +25% typical latency is absorbed by the extra SIS host rather than by a worse service level; encrypted migration costs 13 GB at 2.5 GB/s ≈ 5 s per SIS VM.
3. **Experiments:** cite your Task 9 noisy-neighbour graph (victim p95 versus 0/1/3/7 neighbours, SMT on and off). If the victim's p95 rises sharply with a neighbour, that supports separate pools; if A or C showed little rise, state that the remaining argument is regulatory and the memory/LLC channels. Use your measured mitigation costs in place of the assumed 20%.
4. **Regulatory evidence to produce:** exported VRF/ACL configuration; failed reachability tests from external to SIS (`nmap`, `ping`); host-pool membership and attestation records; separate storage namespaces and key IDs; audit logs of migrations; a penetration-test report of the external pool.
5. **Cost trade-off:** one extra host and under-used SIS hosts buy a provable boundary; the alternative (shared hosts with QoS) is cheaper but cannot show the regulator separation.

12.10 Conclusion

The original design met its performance target only for trusted, small-I/O tenants. With external big-data tenants, network and storage saturate and isolation must be provable, so the design is **modified**, not retained and not rebuilt from scratch: hardened KVM stays for the SIS, and external workloads move to Kata micro-VMs with SR-IOV on their own host pool, network, storage and credentials. The decision follows from the model ($\rho \geq 1$ when mixed), the risk matrix and the regulatory constraint, at the price of one extra host and deliberately low density on the SIS pool.

MARK DISTRIBUTION

Task	Marks
Task 1 – Architecture Characterisation	5
Task 2 – Dominant Overhead Identification	2
Task 3 – Quantitative Performance Model	6
Task 4 – Performance Under Operating Conditions	4
Task 5 – Security Threat and Vulnerability Analysis	3
Task 6 – Isolation–Performance Trade-off Formulation	3
Task 7 – Mitigation Evaluation	4
Task 8 – Implementation and Testing	4
Task 9 – Experimental Performance	3
Task 10 – Theory vs. Experiment	4
Task 11 – Final Architecture Recommendation	4

Subtotal	42
Task 12 – Day-4 Design Challenge	8
TOTAL	**50**